

GRIFFITH UNIVERSITY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY
3806ICT Logic and Automated Reasoning

Trimester 1, 2026

Assignment 2 (50%)

Due Time: 23:59 Friday, 29th May 2026

Assignment overview and requirements

This is a group assignment.

In this assignment, each group of students is expected to develop an LLM agent that interacts with the Isabelle/HOL theorem prover to perform automated reasoning tasks. The LLM-powered theorem prover should have the following features:

- Stepwise prover (in the prover folder)
 - LLM guesses tactics
 - Combined with nitpick, quickcheck, and sledgehammer
 - Beam search for suitable tactics
 - ML reranker for tactics
 - Premise selection using encoders and transformers
- Isar-style proof outline generator (in the planner folder)
 - LLM guesses outline
 - Micro RAG extracted from AFP
 - Calls the stepwise prover to fill the details
 - CEGIS style iterative proof repair

You may build your solution from scratch or build on top of the existing code in this repository:
<https://github.com/zhehou/llm-isabelle>

If you choose to use my repository, note that the “Calls the stepwise prover to fill the details” and the “CEGIS style iterative proof repair” features are marked as *Work In Progress (WIP)*, i.e., they do not function as intended yet, and you should complete these features and make them work properly with good performance.

If you choose to build from scratch, you are still recommended to have a look at my repository to understand the intended design of each feature.

In particular, a design (not *the* design, as you can have your own designs) of the WIP features are described as below.

First, generate an initial proof outline with the LLM. For easy goals this may already be a complete proof (like by simp). For harder goals, the outline should be structured Isar (induction, cases, helper facts), and it may contain sorry placeholders where subproofs are missing.

Next, run Isabelle on the current candidate script. If it passes and there is no sorry, you are done. If it fails, always focus on the earliest failure point (the first error Isabelle reports). This keeps the procedure deterministic and prevents you from fixing symptoms later in the script while something earlier is still broken.

If the earliest failure is a sorry, trigger Fill. Fill means: take that hole, extract the exact subgoal context at that location (assumptions and goal), and call the stepwise prover to produce a small replacement fragment that discharges it. Replace the sorry, then re-run Isabelle. Continue top-down until either all holes are solved or one hole cannot be filled.

If Fill cannot solve a hole, or if Isabelle fails at a non-hole line (a bad have/show, wrong facts, syntax/type issues), trigger Repair. Repair is staged from small to large edits:

- [Stage 1: local repair] start by regenerating only a tiny local step near the failure (a single have/show block).
- [Stage 2: subproof repair] If that does not lead to progress, regenerate the enclosing subproof (a case branch or inner proof ... qed).
- [Stage 3: whole proof repair] If that still does not work, regenerate the full proof.

After any repair edit, run Fill again on any newly introduced sorry placeholders, then re-check with Isabelle. A repair attempt counts as failed if it either cannot be verified (when it contains no sorry) or it introduces holes that Fill cannot discharge.

To stop the system from looping endlessly, use simple budgets: a small number of attempts per repair stage before escalating to the next stage, and a global timeout. Optionally, keep a few alternative candidate scripts and prefer the ones that make Isabelle verify further or reduce the number of remaining holes, rather than always committing to the latest rewrite.

You can be creative and come up with different designs of the WIP features as long as you can justify them.

As in the previous assignment, you will need to

- Find a set of suitable benchmark datasets online or generate some datasets.
 - If you use datasets online, cite the source.
 - If you generate datasets, document the generation process and justify that the dataset is “suitable”:
 - It should contain a balanced set of easy and hard formulae that can put your method to a serious test.
- Conduct comprehensive evaluation of your method (compare it with the baseline, i.e., the repository as is) using the datasets and document your findings.

Everything not specified here and in the marking rubric is up to your interpretation and implementation. These might include:

- the programming language,
- the libraries and existing code you may use,
- how you improve the baseline,

- how you use AI to help you, except for writing.

Instructions for writing the report

- The report should be in LNCS format. Do not change font size, margin size etc.
 - Word template can be found here:

<https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines>

- LaTeX template can be found on Overleaf:

<https://www.overleaf.com/latex/templates/springer-lecture-notes-in-computer-science/kzwwpvhwnvfj>

- The report should be *no more than 16 pages, including references*.
- Your report should have an abstract.
 - No more than 500 words.
 - Give an overview of the topic and what you have learned.
- Your report should have an introduction section.
 - Describe the background of the topic and why it's important.
 - Give an overview of the state-of-the-art/related methods.
 - Give an overview of your work.
- Your report should have a related work section.
 - Discuss related work in the literature.
 - Choose recent work that represents the state-of-the-art.
 - This section essentially expands the overview in introduction with more details.
 - At the end of this section, give a comparison of your work with the related methods.
- Your report should have a “proposed approach” section.
 - Describe your work in detail.
 - Focus on the conceptual design.
- Your report should have an implementation and experiment section.
 - Describe the implementation details (e.g., programming language, libraries, important programming specifics, but no need to show code unless it is significant.)
 - Describe the experiment setup (e.g., OS, hardware specs, etc.).
 - Discuss and analyse the results.
- Your report should have a discussion/conclusion section at the end.
 - Summarise the most significant ideas you have learned.
- Your report should have a “data availability” section of just one sentence that provides the link to your source code.
- The order of sections is up to you.
- Use figures and tables to present your ideas.

- Vectorised images are preferred. If other images (e.g., screenshots) are used, ensure that they do not look blurry/jagged on a 200+ PPI (e.g., 4K 21-inch or 5K 27-inch) screen.
- Cite existing work, including all the libraries and code you used that are not developed by yourselves.
 - The reference should contain the authors, the paper/project title, the year of publication, and the venue of publication (name of conference/journal).
 - In the case of code or library, venue of publication is optional, but provide URL.
- Acknowledge the usage of generative AI. Document the queries and responses in an Appendix (exclude from page limit). Do not use generative AI for writing.

SUBMISSION

Each group submits ONE PDF copy of the report to L@G (Canvas) / 3806ICT / Assessments / Assignment 2 -> Group Report

Each student submits a PDF copy of the peer review form to L@G (Canvas) / 3806ICT / Assessments / Assignment 2 -> Individual Review Form

Late submissions: marks will be deducted according to university policy.